Informe de Pentesting — INTEGRITY BREACH

Nov 11, 2026 · Josep Sanz

Ficha del laboratorio

Laboratorio comprometido en unos 12 minutos (20:15 → 20:27) recuperando la flag `FLAG{N3tw0rk_Tr4ff1c_L34k}` de una captura de red publicada en el servidor web.

Campo	Valor
Plataforma	whoami-labs.com
Laboratorio	INTEGRITY BREACH (`integrity_breach.tar`)
Objetivo	Network Analysis Lab · `172.17.0.2:80` (Apache 2.4.52, Ubuntu)
Dificultad	Fácil (1 punto)
Fecha	2026-11-11, 20:15 – 20:27 (UTC+1)
Flag	`FLAG{N3tw0rk_Tr4ff1c_L34k}`
Herramientas	Nmap 7.991, feroxbuster 2.13.1, navegador, Wireshark
Evidencias	6 capturas de pantalla (figuras 1–6)

Resumen ejecutivo

Un atacante anónimo puede obtener credenciales válidas sin explotar ningún fallo de software: basta con descargar una captura de tráfico que el servidor web publica en `/files/network_capture.pcap`. Riesgo global **Alto**.

La captura contiene dos sesiones en texto claro: un login HTTP (`admin`) y un login FTP correcto (`ftpuser`). Las dos contraseñas, unidas, forman la flag.

El origen del problema son tres malas prácticas encadenadas: guardar capturas de red en la raíz web, dejar el listado de directorios activo y usar protocolos sin cifrar (HTTP y FTP). Corregir cualquiera de las dos primeras habría impedido el compromiso.

Alcance, autorización y metodología

Pruebas realizadas exclusivamente contra el laboratorio INTEGRITY BREACH de whoami-labs.com, desplegado localmente en Docker (`172.17.0.2`) desde Kali Linux (`172.17.0.1`). La plataforma autoriza expresamente el ataque a sus laboratorios.

- **Dentro del alcance:** el host `172.17.0.2` y todos sus servicios.
- **Fuera del alcance:** cualquier otro sistema; sin ataques de denegación de servicio.
- **Metodología:** reconocimiento de puertos → enumeración de contenido web → descarga de ficheros expuestos → análisis forense del tráfico → validación de la flag.

Cadena de ataque

Cuatro pasos sin autenticación llevan de un puerto abierto a credenciales en claro (MITRE ATT&CK T1552 *Unsecured Credentials* y T1040 *Network Sniffing*).

1. **Reconocimiento:** Nmap revela un único servicio, HTTP en el puerto 80.
2. **Enumeración web:** feroxbuster descubre `/files/` con listado de directorios y el fichero `network_capture.pcap`.
3. **Exfiltración:** descarga directa de la captura (3,8 KB) sin autenticación.
4. **Análisis del tráfico:** Wireshark muestra un login HTTP y un login FTP en texto claro; sus contraseñas componen la flag.

Desarrollo de la intrusión

1. Reconocimiento de puertos

Escaneo completo de los 65.535 puertos TCP con detección de versiones y scripts por defecto:

```
nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2
```

Solo el puerto **80/tcp** está abierto: Apache httpd 2.4.52 (Ubuntu), con título *Network Analysis Lab*. El título ya sugiere que el reto gira en torno al análisis de tráfico.



Figura 1 — Resultado de Nmap: único servicio HTTP en el puerto 80.

2. Enumeración de contenido web

Fuzzing de directorios y ficheros con extensiones habituales:

```
feroxbuster -u http://172.17.0.2 -t 5 -x php,bak,txt,html,py,zip \
  -w /usr/share/wordlists/seclists/Discovery/Web-Content/raft-medium-
directories.txt
```

feroxbuster encuentra `/index.html`, el directorio `/files/` (301) y, dentro, **`/files/network_capture.pcap`** (200, 4737 bytes). También avisa de que `/files/` tiene el listado de directorios activo.

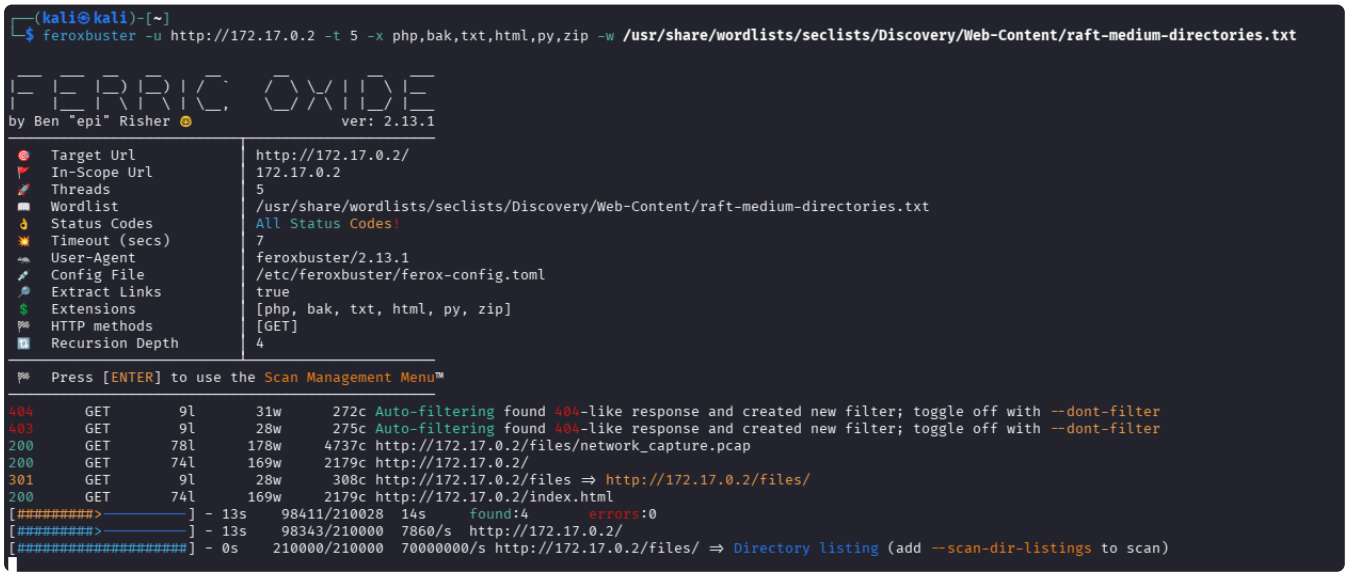


Figura 2 — feroxbuster descubre el fichero `network_capture.pcap`.

3. Acceso al fichero expuesto

Al abrir `http://172.17.0.2/files/` en el navegador aparece el índice automático de Apache (*Index of /files*). Lista el fichero `network_capture.pcap`, de 3,8 KB, modificado el 2026-01-20 16:12. Se descarga sin ninguna autenticación.

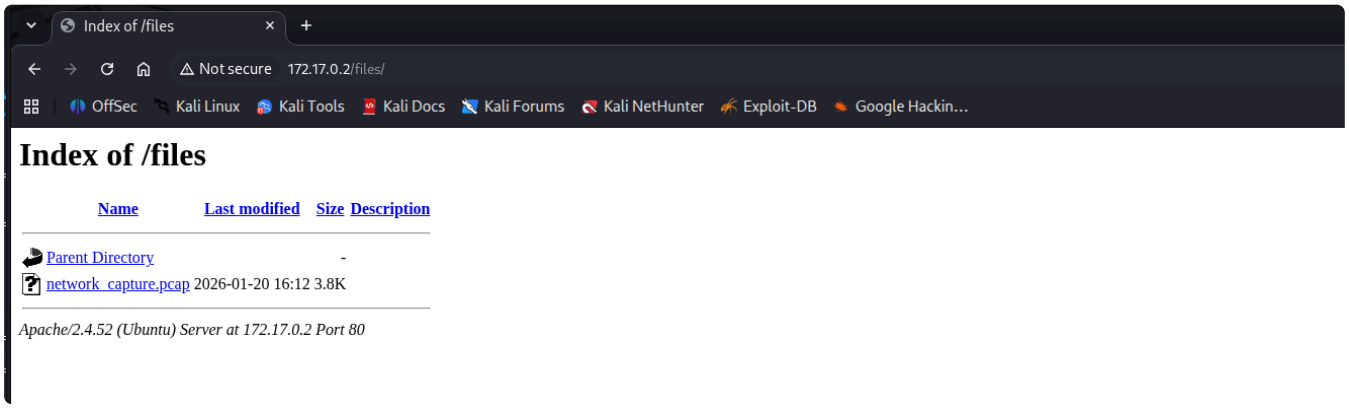


Figura 3 — Listado de directorios de `/files/` con la captura de red descargable.

4. Análisis de la captura: sesión HTTP

La captura contiene 20 paquetes en la interfaz de loopback (`127.0.0.1`). El paquete 4 es un `POST /login.php` enviado con `curl/7.81.0` en formato `application/x-www-form-urlencoded`. El formulario viaja en claro:

- `username` = `admin`
- `password` = `FLAG{N3tw0rk_`

El servidor responde `404 Not Found`, pero la primera mitad de la flag ya es visible.

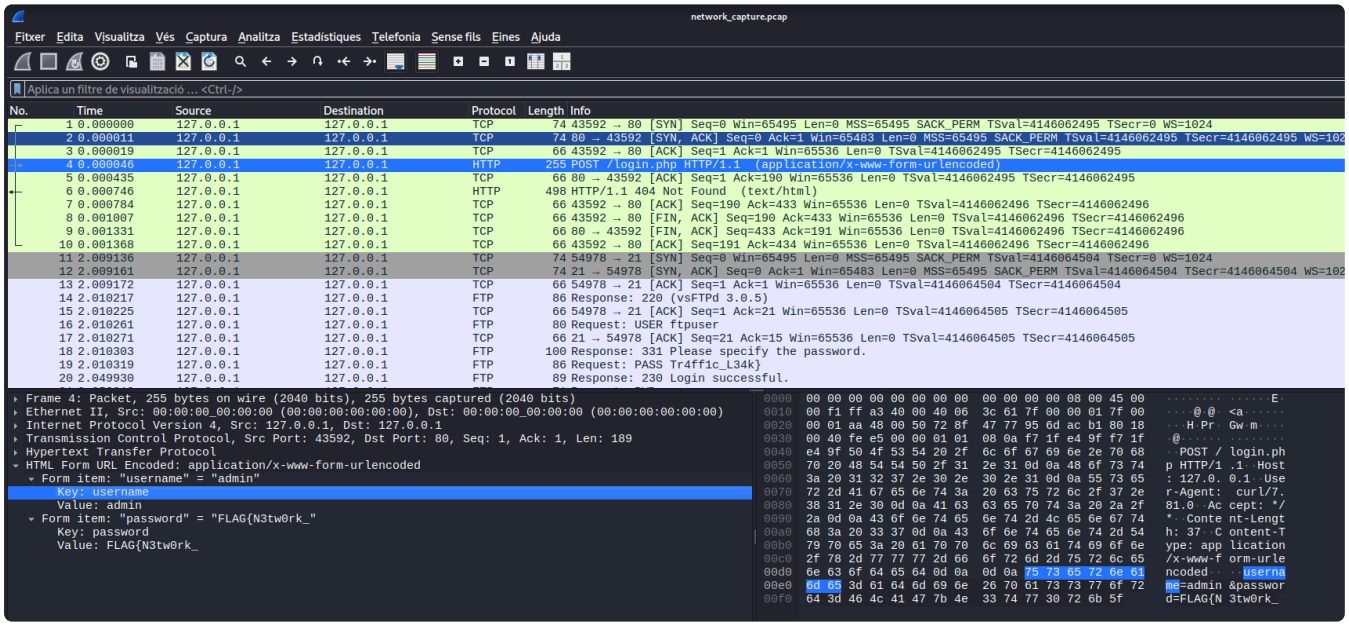


Figura 4 — Wireshark: credenciales del formulario HTTP en texto claro (paquete 4).

5. Análisis de la captura: sesión FTP

Dos segundos después se abre una sesión FTP contra vsFTPd 3.0.5 (puerto 21). Los paquetes 16–20 muestran la autenticación completa en claro:

- `USER ftpuser` → `331 Please specify the password.`
- `PASS Tr4ff1c_L34k}` → `230 Login successful.`

Esta contraseña es **válida** (login correcto) y contiene la segunda mitad de la flag.

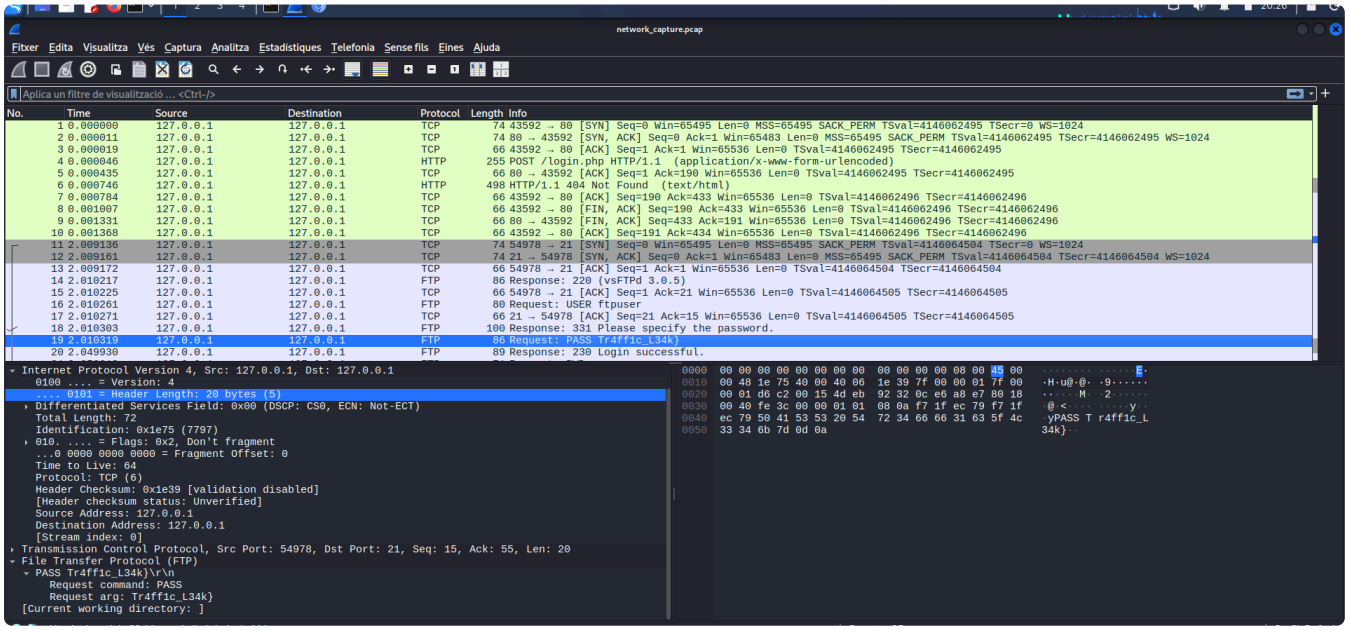


Figura 5 — Wireshark: contraseña FTP en texto claro y login aceptado (paquete 19).

6. Validación de la flag

Uniendo las dos contraseñas se obtiene `FLAG{N3tw0rk_Tr4ff1c_L34k}`. Los tres primeros envíos (solo la primera mitad) fueron rechazados; el cuarto completó el laboratorio a las 20:27:08.



Figura 6 — Flag aceptada: «¡Felicitaciones! Has completado Integrity Breach».

Vulnerabilidades

Cuatro hallazgos; el más grave (CVSS 7.5) es la captura de red publicada sin control de acceso. Puntuaciones con CVSS 3.1.

#	Vulnerabilidad	CWE / OWASP	CVSS 3.1	Severidad
1	Captura de red con credenciales expuesta en la raíz web (`/files/network_capture.pcap`)	CWE-538 · CWE-552 · A01:2021	7.5 · `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`	Alta
2	Credenciales transmitidas en texto claro por HTTP y FTP	CWE-319 · CWE-522 · A02:2021	5.9 · `AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N`	Media
3	Listado de directorios activo en `/files/`	CWE-548 · A05:2021	5.3 · `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`	Media
4	Divulgación de versiones (Apache 2.4.52, vsFTPd 3.0.5, Ubuntu) y servicio sin TLS	CWE-200 · A05:2021	5.3 · `AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`	Media

Impacto. La credencial `ftpuser` / `Tr4ff1c_L34k}` fue aceptada por el servidor FTP de la captura. Si sigue vigente, o se reutiliza en otros servicios, da acceso directo a datos de la organización. La captura también revela la ruta `/login.php` y el usuario `admin`, útiles

para ataques posteriores.

Remediación

Prioridad inmediata: retirar la captura del servidor y rotar las credenciales que contiene.

1. **Retirar y rotar (vuln. 1).** Eliminar `network_capture.pcap` de la raíz web; cambiar las contraseñas de `admin` y `ftpuser`. Guardar capturas y material forense fuera del `DocumentRoot`, en almacenamiento con control de acceso.
2. **Cifrar el tráfico (vuln. 2).** Sustituir FTP por SFTP o FTPS y servir la web solo por HTTPS con HSTS. Ninguna credencial debe viajar en claro, ni siquiera en loopback.
3. **Desactivar el listado de directorios (vuln. 3).** En Apache, `Options -Indexes` en la configuración global o del `<Directory>`; restringir `/files/` con autenticación si debe existir.
4. **Reducir la información expuesta (vuln. 4).** `ServerTokens Prod` y `ServerSignature Off` en Apache; `ftpd_banner` genérico en vsFTPd.
5. **Prevención.** Revisiones periódicas del contenido publicado (p. ej. con feroxbuster o un escáner DAST) y política que prohíba dejar artefactos de diagnóstico (`.pcap`, `.bak`, volcados) en servidores de producción.

Conclusiones

El laboratorio se resolvió sin exploits ni fuerza bruta: el propio servidor entregó las credenciales. Es un fallo de **gestión de secretos**, no de código.

Lección aprendida: una captura de tráfico es tan sensible como una base de datos de contraseñas, porque los protocolos sin cifrar lo registran todo. Dos principios rotos lo explican: *no publicar artefactos sensibles* y *no transmitir credenciales en claro*. Cumplir solo el primero habría cortado la cadena; cumplir el segundo habría dejado la captura sin valor para el atacante.

A nivel de método, la flag estaba dividida entre dos protocolos: conviene revisar **todos** los flujos de una captura (Wireshark → *Follow TCP Stream*) antes de dar por buena una respuesta parcial.